Presentation

지능2

AI 비전 감시 간이 시스템 구축

초 B-3

오 건
김강경
전수현
최유민

Contribution

· 오 건

- Project Manager
- Object Detection
- Flask Video & DB
- AMR Controller

· 전 수 현

- Cctv.pt Labeling & Learning
- AMR Controller
- Report Confirm
- PPT

· 김 강 경

- Flask Templates Layout
- Robot.pt Labeling & Learning
- AMR Controller
- AMR Waypoint

· 최 유 민

- Business Requirement
- System Requirement
- System Design
- Detailed Design

Expo Robot

· 인건비 절감

반복 업무를 대체해 인력 효율성을 높이고 인건비를 절감

· 홍보 효과 상승

관람객의 관심을 끌어 홍보 효과를 극대화

· 데이터 수집

관람객의 관심사와 행동 데이터를 수집해 맞춤형 서비스와 마케팅 전략 수립에 기여

· 브랜드 이미지 제고

혁신적이고 미래지향적인 브랜드 이미지를 강화

Business Requirement

· 군집도 인지	실시간으로 관람객 밀집도를 분석하여 주요 구역 식별
· 자율 이동	목표 지점까지 스스로 이동 및 지속적인 운영(충전 자동 복귀)
· 객체 인지 및 트래킹	관람객 또는 특정 객체를 추적 및 상호작용
· 관리자 인터페이스	홍보 콘텐츠 관리, AMR 상태 모니터링, 비상 정지 기능 제공

System Requirement

▪ 시스템 개요	구성 요소 : 군집도 인지, 네비게이션, 객체 인식 및 트래킹, 관리자 인터페이스
▪ 하드웨어 및 OS 요구사항	PC : Ubuntu 22.04, USB 카메라, Wi-Fi 연결 AMR : Jetson-Orin 프로세서, LiDAR, 최소 2시간 배터리 충전 스테이션 : 20% 이하 시 자동 이동, 수동 배터리 교체
▪ 소프트웨어 요구사항	PC : Python3, ROS2, OpenCV, YOLO(Ultralytics), Flask, SQLite3 AMR : ROS2, OpenCV, SLAM, YOLO(Ultralytics)
▪ 주요 기능	실시간 데이터 처리 : 군집도 분석, AMR 위치 좌표화(SLAM) 저장 및 보관 : 이벤트 정보(DB 저장) 관리자 인터페이스 : 실시간 모니터링, 비상 정지, 이벤트 로그 디스플레이

System Design

- 데이터 흐름 설계

데이터 수집 - CCTV : 군집도 분석을 위해 관람객 위치와 숫자 데이터를 중앙 서버로 송신
AMR 카메라 : 사람 객체 인식 및 추적 데이터 생성
LiDAR : 움직임이 많은 대상을 추적하며 거리 조정

데이터 처리 - 중앙 서버 : YOLO 및 OpenCV로 CCTV 데이터를 분석 → 군집도가 높은 구역 계산 → AMR에 경로 지시
AMR : Jetson-Orin에서 로컬 데이터 처리(객체 인식, 추적)

데이터 저장 - SQLite3 : 이벤트 정보(DB에 객체 수, 이름, 발생 시간 기록)

System Design

CCTV 군집도 인지 - OpenCV와 YOLO로 구역별 관람객 수를 계산 → 중앙 서버로 송신
군집도가 가장 높은 구역 결정

AMR 네비게이션 및 객체 트래킹 -
네비게이션 : SLAM과 LiDAR를 활용해 목표 지점으로 자율 이동
객체 트래킹 : 활동량 많은 대상을 선택
일정 거리 접근 시 정지 → 홍보 콘텐츠 재생

관리자 인터페이스 -
실시간 모니터링(CCTV 및 AMR 카메라 영상)
비상 정지 기능
이벤트 로그 확인(SQLite 기반)

Detailed Design

▪ 상세 설계

데이터 수집 - CCTV : 실시간 영상 → 관람객 수, 구역 좌표 (/cctv/crowd_density)
AMR 카메라 : 영상(/camera/image_raw) → 객체 ID, 위치 좌표(/detected_objects)
LiDAR : 거리 정보(/scan) → 장애물 위치, 추적 대상 거리(/odom)

데이터 처리 - 중앙 서버 : 군집도 분석 → 목표 좌표 발행(/amr/goal_position)
AMR : SLAM 경로 계획(/plan), 객체 인식/추적(/detected_objects)

데이터 저장 - SQLite3 : 군집도 데이터(구역 ID, 객체 수, 시간)
추적 데이터(객체 ID, 좌표, 거리)

Detailed Design

CCTV 군집도 인지 - Input : 실시간 영상 스트림
Process : OpenCV + YOLO로 군집도 분석
Output : 군집도가 높은 구역 좌표(/cctv/crowd_density)

AMR 네비게이션 및 객체 트래킹 - Input : 목표 좌표(/amr/goal_position), 카메라 및 LiDAR 데이터
Process : SLAM 기반 경로 생성, 장애물 회피, YOLO로 객체 추적
Output : 경로 데이터(/plan), 홍보 콘텐츠 재생 트리거(/cmd_vel)

관리자 인터페이스 - Input : ROS 2 상태 데이터(/amr/status), 이벤트 로그(SQLite3)
Output : 실시간 모니터링(CCTV + AMR), 이벤트 로그, 비상 정지 명령(/amr/stop)

Layout

System Alert

- Object Detection - Cctv
  (Count >= 3,
  Confidence >= 0.85)
- Object Detection - AMR

System Monitor

- Display Cctv Camera & Info
- Display AMR Camera & Info
- Real Time
- Class Info
  (Name,
  Count,
  Confidence)
- AMR Position
- Emergency Stop

AMR Controller

- Movement (Slam)
- Navigation (Nav2)
- Object Tracking & Following
  (PID, Pure Pursuit, DWA)

Before

- 관람객을 객체 인식
- 여러 Cctv 화면 중 통행량이 가장 많은 화면의 위치로 한 번에 이동
- Eye Tracking으로 관람객이 태블릿을 응시하는지 확인
- Tracking & Following 후 안내 & 홍보 영상 재생
- AMR의 배터리가 20% 이하일 시 초기 위치로 복귀

After

- RC카와 음료수 병을 객체 인식
- 하나의 Cctv 화면에서 통행량이 많아지면 화면의 위치로 순서대로 이동
- 구현 불가능
- Tracking & Following 후 AMR이 한 바퀴 제자리 회전
- 구현 불가능

Preview

Preview

```
expo_bot
├── __init__.py
├── amr_controller
│   ├── amr_control_node.py
│   ├── navigate_to_goal_node.py
│   └── video_playback_node.py
├── cctv_detection_node.py
├── sys_monitor_node.py
└── templates
    └── two_cam_index.html
```

```
This project is for implemeting amr service robot promoting company in the convention.

#패키지 빌드

colcon build --packages-select expo_bot

#실행하는 순서

#cctv yolo 객체 인식 시작, 이미지 압축하여 발행, 객체인식한 정보 발행, 목표좌표 발행

1. ros2 run expo_bot cctv_detection_node my_best.pt

#AMR봇 자체 좌표 및 영상 토픽으로 발행

2. ros2 run expo_bot amr_control_node

#목표좌표(리스트)와 AMR봇좌표를 구독받고 목표좌표로 네비게이션

3. ros2 run expo_bot navigate_to_goal_node

#모니터링 시스템에서 영상 디스플레이, 구독받은 객체정보 DB 에 저장

4. ros2 run expo_bot sys_monitor_node
```

cctv_detection_node

cctv_detection_node

```python
class CCTVCrowdDetectionNode(Node):
    def __init__(self):
        super().__init__('cctv_detection_node')  # 노드 이름 설정

        qos_profile = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,  # BEST_EFFORT 또는 RELIABLE
            depth=10
        )

        self.crowd_image_publisher_ = self.create_publisher(CompressedImage, '/crowd_image', qos_profile)  #CompressedImage 이미지 퍼블리셔 생성
        self.detected_object_publisher_ = self.create_publisher(String, '/detected_object', qos_profile) #객체인식된 정보 퍼블리셔 생성

        self.json_data = []  # 데이터를 저장할 리스트 초기화
        self.bridge = CvBridge()  # CvBridge 인스턴스 생성S
        self.blue_box_margin = 50  # 파란색 바운딩 박스의 여백 (픽셀 단위)
        self.model = YOLO('./src/expo_bot/resource/cctv.pt')  # YOLO 모델 로드
        self.cap = cv2.VideoCapture(0)  # 카메라 사용
        self.last_alert = None  # 마지막 알림 시간 추적 (초기화)

        self.json_data = []  # JSON 파일에 저장할 데이터를 위한 리스트
```

cctv_detection_node

```python
# 물체가 3개 이상인 경우에만 이미지 저장
if object_count >= 3 and confidence > 0.7:

    # JSON 파일에 객체 정보 저장
    self.json_data.append({
        'class': class_name,
        'confidence': confidence,
        'object_count': object_count,
        'zone': 1,
        'time': current_time
    })

    # JSON 데이터를 문자열로 변환 후 퍼블리시
    json_str = json.dumps(self.json_data)  # JSON 문자열로 변환
    msg = String()  # String 메시지 객체 생성
    msg.data = json_str
    self.detected_object_publisher_.publish(msg) # 객체 인식된 정보 퍼블리셔 생성
```

cctv_detection_node

cctv_detection_node

```
# 이미지를 ROS2 이미지 메시지로 변환하여 퍼블리시
# 이미지 압축 후 퍼블리시
_, encoded_image = cv2.imencode('.jpg', img)  # 이미지 JPEG 형식으로 압축
image_message = CompressedImage()
image_message.header.stamp = self.get_clock().now().to_msg()  # to_msg()로 Time 객체 변환
image_message.format = "jpeg"
image_message.data = np.array(encoded_image).tobytes()  # 압축된 이미지를 바이트로 변환

#image_message = self.bridge.cv2_to_imgmsg(img, encoding="bgr8")
self.crowd_image_publisher_.publish(image_message)
```

sys_monitor_node

```python
class SysMonitorNode(Node):
    def __init__(self):
        super().__init__('sys_monitor_node')

        # 데이터베이스 접근 스레드 안전을 위한 락
        self.db_lock = threading.Lock()

        qos_profile = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,  # BEST_EFFORT 또는 RELIABLE
            depth=10
        )

        # ROS2 토픽 구독 설정
        self.subscription1 = self.create_subscription(
            CompressedImage, 'crowd_image', self.crowd_image_callback, 10)
        self.subscription2 = self.create_subscription(
            CompressedImage, '/amr_camera_image', self.amr_image_callback, qos_profile)
        self.json_subscription = self.create_subscription(
            String, '/detected_object', self.json_callback, 10)

        #목표좌표 퍼블리셔
        self.target_coordinate_publisher_ = self.create_publisher(Point, '/target_coordinate', qos_profile)
```

sys_monitor_node

sys_monitor_node

```
<title>System Monitoring</title>
<script>
    // Function to fetch and update data every 5 seconds
    async function fetchDetectedData() {
        try {
            const response = await fetch('/get_detected_data');
            const data = await response.json();
            const tableBody = document.getElementById('data-table-body');

            tableBody.innerHTML = '';  // Clear current rows

            data.forEach(item => {
                const row = document.createElement('tr');

                // Check if confidence is >= 0.9 and apply red font color if true
                const className = item.class_name;
                const confidence = item.confidence;
                const objectCount = item.object_count;
                const zone = item.zone;
                const time = item.time;

                const confidenceStyle = confidence >= 0.9 ? 'color: red;' : ''; // Apply red color

                row.innerHTML = `<td>${className}</td>
                                 <td style="${confidenceStyle}">${confidence}</td>
                                 <td>${objectCount}</td>
                                 <td>${zone}</td>
                                 <td>${time}</td>`;
                tableBody.appendChild(row);
            });
        } catch (error) {
            console.error("Error fetching data:", error);
        }
    }

    // Refresh data every 5 seconds
    setInterval(fetchDetectedData, 5000);
```

amr_control_node

amr_control_node

```python
class AMRControlNode(Node):
    def __init__(self):
        super().__init__('amr_control_node')

        qos_profile = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,  # BEST_EFFORT 또는 RELIABLE
            depth=10
        )

        # 현재 위치를 발행할 토픽 생성
        self.position_publisher_ = self.create_publisher(Point, '/amr_current_position', qos_profile)

        # 카메라 영상을 발행할 토픽 생성
        self.image_publisher_ = self.create_publisher(CompressedImage, '/amr_camera_image', qos_profile)
        self.bridge = CvBridge()  # OpenCV 브리지 생성

        # 카메라 초기화
        for device_index in range(2):  # 최대 2개의 카메라 확인
            self.cap = cv2.VideoCapture(device_index)
            if self.cap.isOpened():
                self.get_logger().info(f"카메라 초기화 성공: /dev/video{device_index}")
                break
        else:
            self.get_logger().error("카메라 초기화 실패! 연결된 카메라를 확인하세요.")
            return

        # YOLO 모델 로드
        model_path = '/home/rokey9/chob3/chob3_ws/src/expo_bot/resource/robot.pt'
        if not os.path.exists(model_path):
            self.get_logger().error(f"YOLO 모델 파일이 없습니다: {model_path}")
        else:
            self.model = YOLO(model_path)
            #self.model = YOLO('./src/expo_bot/resource/robot.pt')  # YOLO 모델 로드
```

amr_control_node

amr_control_node

```python
def run_detection(self):
    while True:
        # 카메라에서 프레임 읽기
        ret, img = self.cap.read()
        if not ret or img is None:
            self.get_logger().error("카메라 프레임을 읽을 수 없습니다. 연결 상태를 확인하세요.")
            return  # 비정상적인 경우 처리 중단

        # YOLO 모델 적용
        try:
            results = self.model(img, stream=True)
            # (이하 YOLO 객체 인식 처리 코드)
        except Exception as e:
            self.get_logger().error(f"YOLO 처리 중 오류 발생: {e}")
            return

        for r in results:
            boxes = r.boxes  # 감지된 객체들의 바운딩 박스 정보

            for box in boxes:
                # 바운딩 박스 좌표 추출
                x1, y1, x2, y2 = map(int, box.xyxy[0])
                confidence = float(box.conf[0].item())  # 객체의 정확도 (Tensor -> float 변환)
                class_id = int(box.cls[0].item())  # 객체의 클래스 ID (Tensor -> int 변환)

                # 클래스 이름을 가져오기 (예: 'car', 'person', etc.)
                class_name = self.model.names[class_id]

                # 객체의 바운딩 박스를 화면에 그림
                cv2.rectangle(img, (x1, y1), (x2, y2), (0, 0, 255), 2)
                # 클래스 이름과 confidence를 표시
                label = f"{class_name} {confidence:.2f}"
                cv2.putText(img, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 2)

        # 이미지를 ROS2 이미지 메시지로 변환하여 퍼블리시
        # 이미지 압축 후 퍼블리시
        _, encoded_image = cv2.imencode('.jpg', img)  # 이미지 JPEG 형식으로 압축
        image_message = CompressedImage()
        image_message.header.stamp = self.get_clock().now().to_msg()  # to_msg()로 Time 객체 변환
        image_message.format = "jpeg"
        image_message.data = np.array(encoded_image).tobytes()  # 압축된 이미지를 바이트로 변환

        #image_message = self.bridge.cv2_to_imgmsg(img, encoding="bgr8")
        self.image_publisher_.publish(image_message)
```

navigate_to_goal_node

```python
class NavigateToGoalNode(Node):
    def __init__(self):
        super().__init__('navigate_to_goal_node')

        # BasicNavigator 초기화
        self.navigator = BasicNavigator()

        # 초기 위치 설정
        self.set_initial_pose()

        # Nav2 활성화 대기
        self.navigator.waitUntilNav2Active()
        self.get_logger().info("Nav2가 활성화되었습니다.")

        # 목표 좌표와 현재 좌표 초기화
        self.target_positions = []  # 목표 좌표 리스트
        self.current_position = Point()  # AMR 현재 위치 저장

        qos_profile = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,  # BEST_EFFORT 또는 RELIABLE
            depth=10
        )

        # /target_coordinate 토픽 구독 (목표 좌표 수신)
        self.create_subscription(Point, '/target_coordinate', self.target_callback, qos_profile)

        # /amr_current_position 토픽 구독 (현재 위치 수신)
        self.create_subscription(Point, '/amr_current_position', self.amr_position_callback, qos_profile)
```

```yaml
image: /home/rokey9/chob3/tb_ws_map.pgm
mode: trinary
resolution: 0.05
origin: [-2.24, -1.64, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

navigate_to_goal_node

```python
def set_initial_pose(self):
    """맵 프레임에서 초기 위치 설정"""
    initial_pose = PoseStamped()
    initial_pose.header.frame_id = 'map'  # 맵 좌표계
    initial_pose.header.stamp = self.get_clock().now().to_msg()
    initial_pose.pose.position.x = 0.0
    initial_pose.pose.position.y = 0.0

    # 오일러 각도 -> 쿼터니안 변환
    roll = 0.0
    pitch = 0.0
    yaw = 0.1178  # 약 6.75도 (라디안 값)
    quat = self.euler_to_quaternion(roll, pitch, yaw)
    initial_pose.pose.orientation.x = quat[0]
    initial_pose.pose.orientation.y = quat[1]
    initial_pose.pose.orientation.z = quat[2]
    initial_pose.pose.orientation.w = quat[3]

    self.navigator.setInitialPose(initial_pose)
    self.get_logger().info("초기 위치가 설정되었습니다.")
```

navigate_to_goal_node

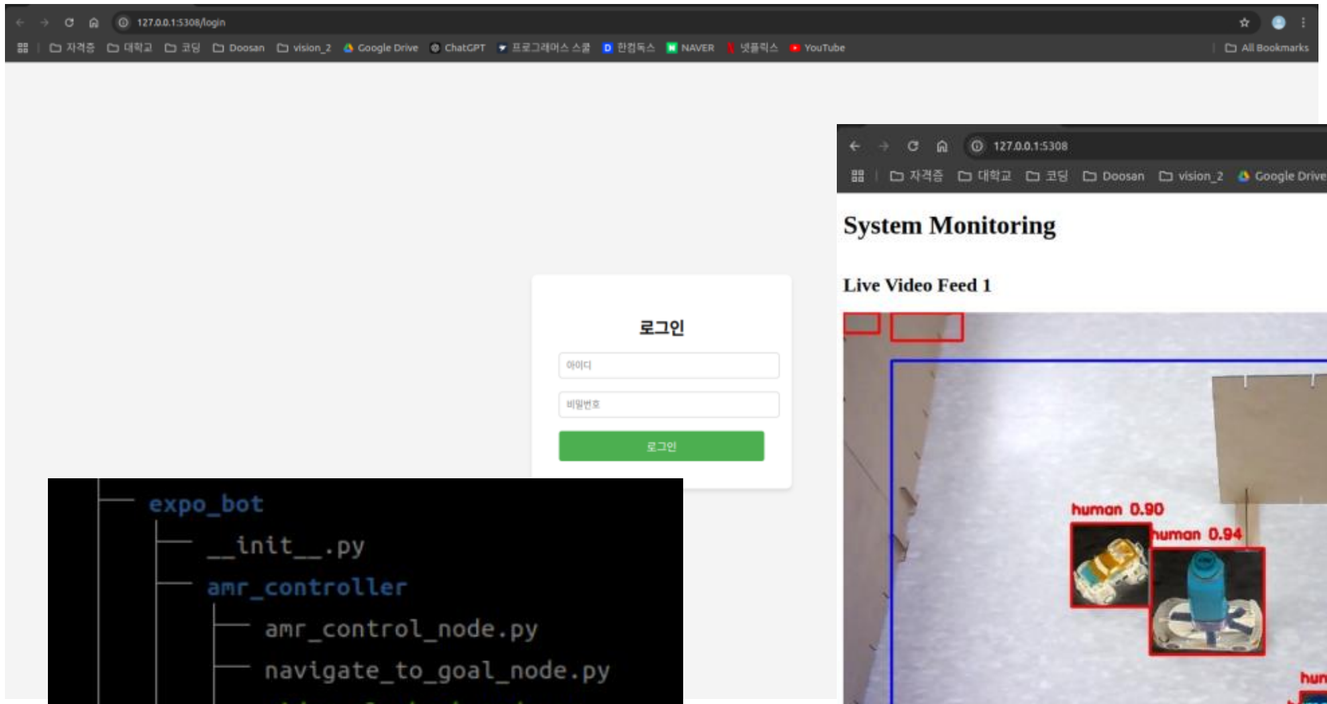
navigate_to_goal_node

```python
def move_to_next_goal(self):
    """목표 좌표 리스트에서 다음 위치로 이동"""
    if len(self.target_positions) == 0:
        self.get_logger().info("이동할 목표 좌표가 없습니다.")
        return

    # 목표 좌표 설정
    target = self.target_positions.pop(0)
    goal_pose = PoseStamped()
    goal_pose.header.frame_id = 'map'
    goal_pose.header.stamp = self.get_clock().now().to_msg()
    goal_pose.pose.position.x = target.x
    goal_pose.pose.position.y = target.y

    # 방향 설정 (오일러 -> 쿼터니안)
    roll = 0.0
    pitch = 0.0
    yaw = 0.0  # 목표 지점에서의 방향 (0으로 기본값 설정)
    quat = self.euler_to_quaternion(roll, pitch, yaw)
    goal_pose.pose.orientation.x = quat[0]
    goal_pose.pose.orientation.y = quat[1]
    goal_pose.pose.orientation.z = quat[2]
    goal_pose.pose.orientation.w = quat[3]

    self.get_logger().info(f"목표 좌표로 이동 시작: x={target.x}, y={target.y}")

    # 목표 좌표를 /move_base_simple/goal 토픽에 발행
    self.goal_publisher.publish(goal_pose)
    self.get_logger().info(f"목표 좌표 발행: x={target.x}, y={target.y}")

    self.navigator.goToPose(goal_pose)
```

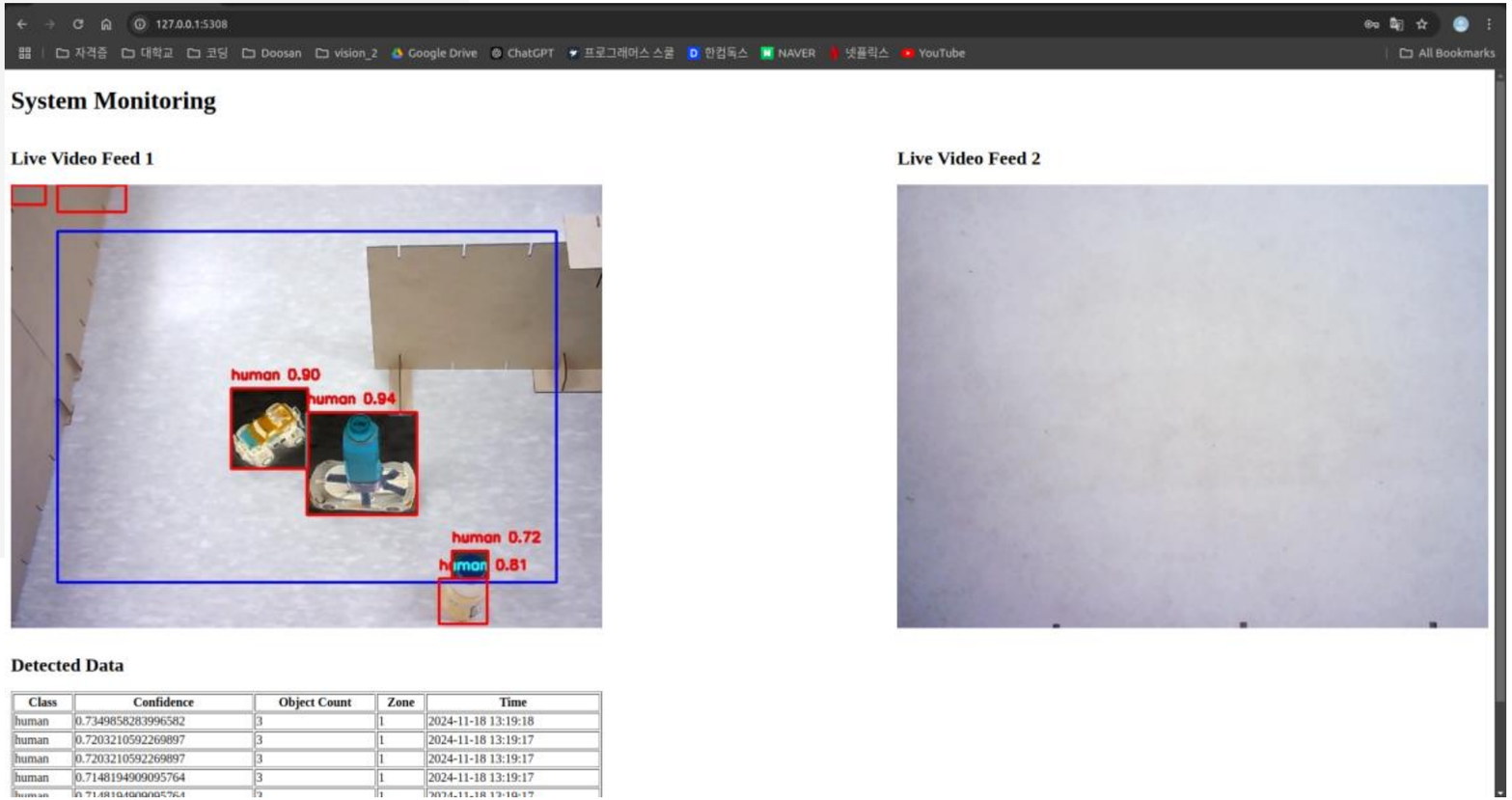
navigate_to_goal_node

```python
# 이동 완료 여부 확인
while not self.navigator.isTaskComplete():
    feedback = self.navigator.getFeedback()
    if feedback:
        distance_to_goal = ((goal_pose.pose.position.x - self.current_position.x) ** 2 +
                            (goal_pose.pose.position.y - self.current_position.y) ** 2) ** 0.5
        self.get_logger().info(f"목표까지 남은 거리: {distance_to_goal:.2f}m")
        if distance_to_goal < 0.2:  # 예시로 20cm 이하로 가까워졌을 때
            self.get_logger().info("목표에 충분히 근접하여 이동을 종료합니다.")
            break

# 이동 결과 확인
result = self.navigator.getResult()
if result == 1:  # 기본값 1이 성공을 의미
    self.get_logger().info(f"목표 좌표에 도달했습니다: x={target.x}, y={target.y}")
else:
    self.get_logger().info("목표 좌표에 도달하지 못했습니다.")

# 다음 목표 좌표로 이동
self.move_to_next_goal()
```

Conclusion




```
├── expo_bot
│   ├── __init__.py
│   ├── amr_controller
│   │   ├── amr_control_node.py
│   │   ├── navigate_to_goal_node.py
│   │   └── video_playback_node.py
│   ├── cctv_detection_node.py
│   ├── sys_monitor_node.py
│   └── templates
│       └── two_cam_index.html
```



Class	Confidence	Object Count	Zone	Time
human	0.7349858283996582	3	1	2024-11-18 13:19:18
human	0.7203210592269897	3	1	2024-11-18 13:19:17
human	0.7203210592269897	3	1	2024-11-18 13:19:17
human	0.71481949090957 64	3	1	2024-11-18 13:19:17

Thank You

초 B-3
오 건, 김강경, 전수현, 최유민